

C++ STUDY NOTES

CHAPTER 5

Standard Template Library (STL)

Containers, Algorithms, Iterators and Functors: vector, list, set, multiset, map, multimap, and the `<algorithm>/<numeric>` Toolbox

University : Rani Channamma University, Belagavi

Course Code : 24MSCCS 1.3

Programme : M.Sc. Computer Science

Semester : First Semester

Syllabus : 2024-25 onwards

Author : Sudarshan Badli

Academic Yr : 2026-2027

AUTHORSHIP

Prepared and Created by: Sudarshan Badli

© 2026 Sudarshan Badli. All Rights Reserved.

Table of Contents

5.1 Standard Template Library (STL) — Overview and Components	3
5.2 Containers: Sequence Containers and Associative Containers	4
5.3 Sequence Container: the vector Class	5
5.4 The list Class — Inserting, Deleting and Traversing	6
5.5 The set and multiset Classes	7
5.6 The map and multimap Classes	8
5.7 Iterators	9
5.8 Common Algorithms and Other List Operations	10
5.14 Examination Preparation — Chapter 5	12

HOW THIS CHAPTER IS ORGANISED

- Follows the official RCUB M.Sc. Computer Science syllabus (24MSCCS 1.3), Unit V, exactly — no topic is skipped or compressed.
- Each topic: Definition → Syntax → Key points → Full worked C++ program(s) with output.
- Sections 5.9–5.13 add deep-dive material: a vector dry-run trace table, line-by-line explanation, advantages/disadvantages/applications, common mistakes, and head-to-head container comparisons (vector vs list, set vs map, sequence vs associative).
- Section 5.14 is a complete examination-preparation pack: 2/5/10-mark Q&A, programming questions, output prediction, find-the-error, MCQs and viva questions.
- This is the final chapter of the series, completing all five units of the 24MSCCS 1.3 syllabus.

5.
1

Standard Template Library (STL) — Overview and Components

DEFINITION

The **Standard Template Library** is a collection of generic, reusable, template-based classes and functions supplied with C++, providing ready-made data structures and algorithms so programmers do not need to re-implement common ones (lists, trees, sorting, searching). Because STL components are templates, they work with any data type.

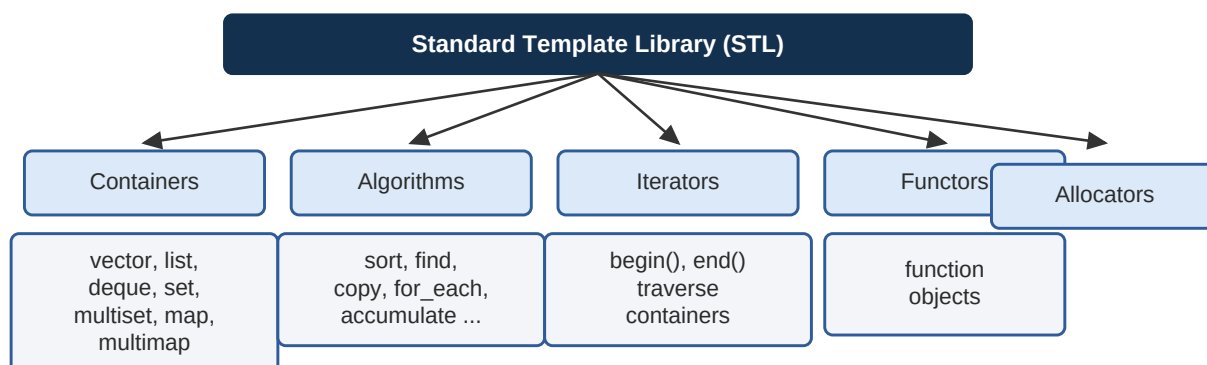
SYNTAX

```
#include <vector>
#include <list>
#include <set>
#include <map>
#include <algorithm>
using namespace std;
```

KEY POINTS

- **Containers:** objects that store collections of data (vector, list, deque, set, map ...).
- **Algorithms:** generic functions that operate on ranges of elements via iterators (sort, find, count, accumulate ...), independent of the specific container.
- **Iterators:** objects that act like generalised pointers, used to traverse a container's elements uniformly (begin() to end()), connecting algorithms to containers.
- **Functors (function objects):** objects that behave like functions (overload operator ()), often used to customise algorithm behaviour (e.g. custom sort order).
- **Allocators:** manage memory allocation for containers (default allocator is used in most programs; rarely customised in coursework).

STL COMPONENTS



PROGRAM – A GLIMPSE OF THE STL STYLE

```
#include <iostream>
using namespace std;

#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {5, 2, 8, 1, 9};
    sort(v.begin(), v.end());           // algorithm + iterators on a container
    for (int x : v) cout << x << " ";
    cout << endl;
    return 0;
}
```

OUTPUT

```
1 2 5 8 9
```

5.2

Containers: Sequence Containers and Associative Containers

DEFINITION

STL containers are classified by how they organise and access elements.

Category	Containers	Characteristic
Sequence	vector, list, deque, array, forward_list	Elements kept in a linear, user-controlled order
Associative	set, multiset, map, multimap	Elements kept sorted automatically by key (usually a balanced tree)
Unordered (hash-based)	unordered_set, unordered_map ...	Elements grouped by hash, average O(1) access, no ordering
Container adaptors	stack, queue, priority_queue	Restricted interface built on top of another container

KEY DIFFERENCE: SEQUENCE VS ASSOCIATIVE

A **sequence container** stores elements in the order you insert them (position-based access, e.g. `v[2]`). An **associative container** stores elements organised by a key, automatically kept sorted, and accessed by that key rather than position (e.g. `m["Asha"]`).

5.3

Sequence Container: the vector Class

DEFINITION

A vector is a dynamic array: contiguous storage that automatically grows as elements are added, with fast random access ($O(1)$ by index) and fast insertion/removal at the end (amortised $O(1)$), but slower insertion/removal in the middle ($O(n)$).

SYNTAX

```
#include <vector>
vector<type> v;           // empty vector
vector<type> v(n);        // n default-valued elements
vector<type> v = {v1, v2, v3}; // initializer list
v.push_back(value); v.pop_back();
v.size(); v.empty(); v[i]; v.at(i);
v.begin(); v.end(); v.insert(pos, val); v.erase(pos);
```

PROGRAM

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {40, 10, 30, 20};
    v.push_back(50); // add at the end
    cout << "Size = " << v.size() << "\n";

    sort(v.begin(), v.end()); // algorithm works via iterators
    cout << "Sorted: ";
    for (int x : v) cout << x << " ";
    cout << "\n";

    v.erase(v.begin()); // remove first element
    cout << "After erase: ";
    for (int x : v) cout << x << " ";
    cout << "\n";
    return 0;
}
```

OUTPUT

```
Size = 5
Sorted: 10 20 30 40 50
After erase: 20 30 40 50
```

5.4

The list Class — Inserting, Deleting and Traversing

DEFINITION

A list is a doubly-linked list: fast insertion/removal anywhere ($O(1)$, given an iterator position) but no random-access indexing (no $l[i]$) — elements must be reached by moving an iterator step by step ($O(n)$ to reach the i -th element).

SYNTAX

```
#include <list>
list<type> l;
l.push_back(value); l.push_front(value);
l.pop_back(); l.pop_front();
l.insert(it, value); l.erase(it);
l.begin(); l.end(); l.remove(value); l.sort(); l.reverse();
```

PROGRAM – INSERTING, DELETING, TRAVERSING A LIST

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> l = {10, 20, 30};
    l.push_front(5);           // insert at front
    l.push_back(40);           // insert at back

    // insert 15 right after 10
    auto it = l.begin();
    ++it;                      // now points to 20
    l.insert(it, 15);

    l.remove(30);              // delete all elements equal to 30

    cout << "List contents: ";
    for (int x : l) cout << x << " "; // traversing the list
    cout << "\n";
    return 0;
}
```

OUTPUT

```
List contents: 5 10 15 20 40
```

5.5 The set and multiset Classes

DEFINITION

A set stores **unique** elements automatically kept in sorted order (by default ascending, using <); inserting a duplicate has no effect. A multiset is identical but **allows duplicate** elements. Both are typically implemented as a balanced binary search tree, giving $O(\log n)$ insert/search/erase.

SYNTAX

```
#include <set>
set<type> s;
s.insert(value); s.erase(value);
s.find(value);    // returns an iterator, or s.end() if not found
s.count(value);   // 0 or 1 for a set; can be >1 for a multiset
multiset<type> ms; // same interface, duplicates allowed
```

PROGRAM

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s;
    s.insert(30); s.insert(10); s.insert(20); s.insert(10); // duplicate 10 ignored

    cout << "Set (sorted, unique): ";
    for (int x : s) cout << x << " ";
    cout << "\n";

    if (s.find(20) != s.end())
        cout << "20 found in the set\n";

    multiset<int> ms = {5, 5, 2, 8, 2};
    cout << "Multiset: ";
    for (int x : ms) cout << x << " ";
    cout << "\ncount of 2 in multiset = " << ms.count(2) << "\n";
    return 0;
}
```

OUTPUT

```
Set (sorted, unique): 10 20 30
20 found in the set
Multiset: 2 2 5 5 8
count of 2 in multiset = 2
```

5.6 The map and multimap Classes

DEFINITION

A map stores **key-value pairs** with **unique** keys, automatically kept sorted by key, giving $O(\log n)$ lookup by key — conceptually an associative array / dictionary. A **multimap** is identical but allows multiple values to share the same key.

SYNTAX

```
#include <map>
map<keyType, valueType> m;
m[key] = value;           // insert or update
m.insert({key, value});
m.find(key); m.erase(key); m.count(key);
for (auto &p : m) { p.first; p.second; } // traverse key/value pairs
```

PROGRAM – STUDENT ROLL-NUMBER TO MARKS MAP

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    map<int, string> students;
    students[101] = "Asha";
    students[103] = "Ravi";
    students[102] = "Meera";           // keys auto-sorted: 101, 102, 103

    cout << "Roll -> Name\n";
    for (auto &p : students)
        cout << p.first << " -> " << p.second << "\n";

    if (students.find(102) != students.end())
        cout << "Found roll 102: " << students[102] << "\n";

    students.erase(101);
    cout << "Size after erase = " << students.size() << "\n";
    return 0;
}
```

OUTPUT

```
Roll -> Name
101 -> Asha
102 -> Meera
103 -> Ravi
Found roll 102: Meera
Size after erase = 2
```

5.7 Iterators

DEFINITION

An **iterator** is an object that behaves like a pointer, used to point to and step through a container's elements without exposing the container's internal implementation. Every STL container provides `begin()` (an iterator to the first element) and `end()` (one position past the last element).

SYNTAX

```
container<type>::iterator it;
for (it = c.begin(); it != c.end(); ++it) {
    cout << *it;           // dereference to get the value, like a pointer
}

// range-based for (uses iterators internally, preferred style)
for (auto &x : c) { cout << x; }
```

KEY POINTS

- Iterator categories (increasing capability): input, output, forward, bidirectional, random-access — a vector supports random-access, a list only bidirectional (no `it + 5` jump).
- `auto` lets the compiler deduce the correct iterator type automatically, avoiding long type names.
- `rbegin()`/`rend()` give reverse iterators for traversing back to front.

PROGRAM

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30, 40};

    cout << "Forward : ";
    for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
        cout << *it << " ";

    cout << "\nReverse : ";
    for (auto it = v.rbegin(); it != v.rend(); ++it)
        cout << *it << " ";
    cout << "\n";
    return 0;
}
```

OUTPUT

```
Forward : 10 20 30 40
Reverse : 40 30 20 10
```

5.
8

Common Algorithms and Other List Operations

DEFINITION

The `<algorithm>` header supplies generic functions that work through iterators on any compatible container, decoupling the algorithm from the specific container used.

SYNTAX

```
#include <algorithm>
sort(first, last);
reverse(first, last);
find(first, last, value);
count(first, last, value);
max_element(first, last); min_element(first, last);
#include <numeric>
accumulate(first, last, initValue);
```

PROGRAM

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;

int main() {
    vector<int> v = {4, 8, 2, 9, 5};

    sort(v.begin(), v.end());
    cout << "Sorted : "; for (int x : v) cout << x << " ";

    cout << "\nMax : " << *max_element(v.begin(), v.end());
    cout << "\nMin : " << *min_element(v.begin(), v.end());
    cout << "\nSum : " << accumulate(v.begin(), v.end(), 0);

    auto it = find(v.begin(), v.end(), 8);
    if (it != v.end()) cout << "\n8 found at position " << (it - v.begin());
    cout << "\n";
    return 0;
}
```

OUTPUT

```
Sorted : 2 4 5 8 9
Max : 9
Min : 2
Sum : 28
8 found at position 3
```

STL Container Quick Comparison

Container	Ordering	Duplicates	Typical access	Underlying structure
vector	Insertion order	Allowed	O(1) by index	Dynamic array
list	Insertion order	Allowed	O(n), sequential	Doubly linked list
set	Sorted by key	Not allowed	O(log n)	Balanced BST
multiset	Sorted by key	Allowed	O(log n)	Balanced BST
map	Sorted by key	Unique keys	O(log n)	Balanced BST

Container	Ordering	Duplicates	Typical access	Underlying structure
multimap	Sorted by key	Duplicate keys allowed	$O(\log n)$	Balanced BST

5.9 Deep Dive — STL: Advantages, Disadvantages, Applications

Advantages, Disadvantages and Applications

ADVANTAGES

- Saves development time: ready-made, thoroughly tested containers and algorithms replace hand-written data structures.
- Generic (template-based): the same vector, sort(), etc. work for int, double, string, or any user-defined type with the required operators.
- Consistent iterator-based interface lets algorithms work across many different container types without modification.
- Highly optimised implementations (e.g. sort() is typically an introsort, combining quicksort, heapsort and insertion sort for best practical performance).

DISADVANTAGES

- Learning curve: template error messages can be long and hard to read for beginners.
- Some overhead compared with a hand-tuned, problem-specific data structure (though usually negligible for typical coursework/application use).
- Choosing the wrong container (e.g. a list when random access is really needed) can hurt performance significantly.
- Modifying a container while iterating over it (e.g. erasing elements in a naive loop) can invalidate iterators and cause subtle bugs.

APPLICATIONS

- Rapid prototyping and competitive programming, where reusable, reliable containers and algorithms save significant coding time.
- Real-world software: compilers, browsers, and large applications use STL-like containers extensively for internal data management.
- Academic and interview contexts: sorting, searching, and data-structure questions are very commonly solved directly with STL.

Line-by-Line Explanation

Line / Statement	Explanation
<code>vector<int> v = {5, 2, 8, 1, 9};</code>	Declares v as a vector of int and initialises it with 5 elements using a C++11 initializer list; internally, vector allocates contiguous dynamic storage for them.
<code>sort(v.begin(), v.end());</code>	Calls the generic sort() algorithm from <algorithm>, passing a range defined by two iterators: v.begin() (the first element) and v.end() (one past the last) — sort() works through these iterators, not through v directly.
<code>for (int x : v) cout << x << " ";</code>	A range-based for loop (internally using v.begin()/v.end()) visits each element of the now-sorted vector in order, printing it.

COMMON MISTAKES / DEBUGGING TIPS

- Using list when random-access indexing (`l[i]`) is needed — list has no `[]` operator; use vector instead if indexing matters.
- Erasing elements from a vector/list while iterating with a simple range-based for loop — this invalidates the iterator mid-loop; use the iterator returned by `erase()` itself to continue safely, e.g. `it = c.erase(it);`.
- Forgetting that map/set keep elements **sorted by key** automatically — insertion order is NOT preserved, which surprises students expecting dictionary-style insertion order.

- Using `v[i]` on an out-of-range index — unlike `.at(i)`, the `[]` operator does **not** perform bounds checking and causes undefined behaviour instead of throwing an exception.
- Comparing two different container types' iterators, or using an iterator after the container has been resized/reallocated — both are common sources of silent, hard-to-debug errors.

5.10 Deep Dive — Dry Run: vector Sort and Search (Topic 5.3)

TRACE TABLE

Step	Operation	Vector contents after the step
1	<code>vector<int> v = {40,10,30,20};</code>	[40, 10, 30, 20]
2	<code>v.push_back(50);</code>	[40, 10, 30, 20, 50]
3	<code>sort(v.begin(), v.end());</code>	[10, 20, 30, 40, 50]
4	<code>v.erase(v.begin());</code> // removes first element	[20, 30, 40, 50]

5.11 Comparison — vector vs list (frequently asked)

Point	vector	list
Underlying structure	Dynamic array (contiguous)	Doubly linked list
Random access (<code>v[i]</code>)	$O(1)$ — fast	Not supported; must walk from <code>begin()</code> ($O(n)$)
Insert/erase at the end	Amortised $O(1)$	$O(1)$
Insert/erase in the middle	$O(n)$ — must shift elements	$O(1)$, given an iterator to the position
Memory overhead per element	Low (just the element)	Higher (two pointers per node, prev/next)
Cache performance	Excellent (contiguous memory)	Poor (nodes scattered in memory)

5.12 Comparison — set vs map (frequently asked)

Point	set	map
Stores	Just keys/values (one item per element)	Key-value PAIRS
Access	By value, via <code>find()</code>	By key: <code>m[key]</code> or <code>m.find(key)</code>
Duplicates	Not allowed (use multiset otherwise)	Keys unique (use multimap otherwise)
Typical use	Membership testing, removing duplicates	Dictionary / lookup-table style data

5.13 Comparison — Sequence vs Associative Containers (summary)

Point	Sequence (vector, list, deque)	Associative (set, map, multiset, multimap)
Order	Insertion order, user-controlled	Automatically sorted by key
Typical lookup cost	$O(n)$ for search (unless indexed, $O(1)$)	$O(\log n)$, via balanced tree
Access style	By position/index	By key/value
When to prefer	Order matters, or frequent end-insertions	Frequent lookups by key, automatic sorting wanted

NOTE ON QUESTION SOURCE

- The questions below are **expected / practice questions** prepared from the official RCUB syllabus for this chapter, in the common style of university exam papers.
- They are **not** claimed to be verified previous-year question papers — cross-check against your department's official previous-year papers for the exact verified questions, if available.

A. Two-Mark Questions and Answers

Q1. [2 marks] What is the Standard Template Library (STL)?

A collection of generic, reusable, template-based classes and functions supplied with C++, providing ready-made containers, algorithms, iterators and functors.

Q2. [2 marks] Name the four main components of STL.

Containers (store data), Algorithms (operate on data via iterators), Iterators (traverse containers), Functors (function objects used to customise algorithm behaviour).

Q3. [2 marks] What is a container in STL?

An object that stores a collection of elements, e.g. vector, list, set, map, providing operations to insert, access and remove elements.

Q4. [2 marks] Differentiate sequence and associative containers in one line.

Sequence containers (vector, list) keep elements in insertion/positional order; associative containers (set, map) keep elements automatically sorted by key.

Q5. [2 marks] What is an iterator?

An object that behaves like a pointer, used to point to and traverse a container's elements without exposing its internal implementation.

Q6. [2 marks] What does `begin()` and `end()` return for a container?

`begin()` returns an iterator to the first element; `end()` returns an iterator to one position PAST the last element (not the last element itself).

Q7. [2 marks] What is the time complexity of accessing an element by index in a vector?

$O(1)$ — constant time, since vector uses contiguous memory, like an array.

Q8. [2 marks] What is the time complexity of searching in a set?

$O(\log n)$, since set is typically implemented as a balanced binary search tree.

Q9. [2 marks] What is the difference between set and multiset?

set stores only unique elements (duplicates are ignored on insert); multiset allows duplicate elements.

Q10. [2 marks] What is the difference between map and multimap?

map requires unique keys; multimap allows multiple values to share the same key.

Q11. [2 marks] What does the `push_back()` function do?

Inserts a new element at the end of a sequence container such as vector or list.

Q12. [2 marks] What does the `sort()` algorithm require of its arguments?

Two iterators marking a range [first, last) that supports random access (such as a vector's iterators) by default, though overloads can sort other ranges with a custom comparator.

Q13. [2 marks] What does `accumulate()` do, and which header provides it?

It sums (or otherwise combines) all elements in a range given a starting value; provided by the header.

Q14. [2 marks] What is a functor?

A function object: a class or struct that overloads `operator()`, allowing an object of that type to be called like a function, often used to customise algorithm behaviour.

Q15. [2 marks] Why does list not support the `[]` operator?

Because list is a doubly linked list, not contiguous memory, so there is no $O(1)$ way to jump directly to the i -th element — only sequential traversal is possible.

B. Five-Mark Questions and Answers

Q1. [5 marks] Explain the components of STL with a diagram and brief description of each.

STL is built from four interacting components. **Containers** are objects that store collections of data, grouped into sequence (vector, list, deque), associative (set, map, multiset, multimap) and container-adaptor (stack, queue, priority_queue) categories. **Algorithms** are generic functions — `sort()`, `find()`, `count()`, `accumulate()` and many more — that operate on a range of elements specified by iterators, independent of which specific container supplies that range. **Iterators** act as generalised pointers that connect algorithms to containers uniformly, via `begin()/end()`, regardless of the container's internal structure. **Functors** (function objects) overload `operator()` so an object can be called like a function, frequently used to customise algorithm behaviour, such as supplying a custom sort order. The component diagram in Topic 5.1 shows how these four pieces interact.

Q2. [5 marks] Explain the vector class with its declaration syntax, common member functions, and a complete program.

`vector` (from `<vector>`) implements a dynamic array: contiguous storage that automatically grows as elements are added, giving $O(1)$ random access by index and amortised $O(1)$ insertion/removal at the end, but $O(n)$ insertion/removal in the middle since subsequent elements must shift. Key operations: `push_back()/pop_back()` add/remove at the end, `size()` and `empty()` query its state, `[]` and `at()` access elements (`at()` additionally bounds-checks and throws `out_of_range`), and `begin()/end()` expose iterators for use with STL algorithms such as `sort()`. The Topic 5.3 program builds a vector of 4 integers, `push_backs` a fifth, sorts it with `sort(v.begin(), v.end())`, and erases the first (smallest) element with `v.erase(v.begin())`, printing the vector's contents at each stage.

Q3. [5 marks] Explain the list class, contrasting it with vector, and demonstrate insertion, deletion and traversal with a program.

`list` (from `<list>`) implements a doubly linked list: each element is a separate node holding pointers to its predecessor and successor, giving $O(1)$ insertion/removal anywhere (given an iterator to the position) but no random-access indexing — reaching the i -th element requires walking node by node, $O(n)$. Key operations include `push_front()/push_back()` to insert at either end, `insert(iterator, value)` to insert before a given position, `erase(iterator)` to remove a specific element, and `remove(value)` to remove all elements equal to a given value. The Topic 5.4 program builds a list, inserts elements at the front and back, inserts a value in the middle using an iterator, removes all occurrences of another value with `remove()`, and traverses the final list with a range-based for loop.

Q4. [5 marks] Discuss the set and multiset classes with their characteristics, and a program demonstrating insertion, searching and counting.

`set` (from `<set>`) stores unique elements, automatically kept in sorted order (ascending by default), typically implemented as a balanced binary search tree giving $O(\log n)$ insert/search/erase; inserting a value already present has no effect. `multiset` is identical but permits duplicate elements. Key operations: `insert(value)`, `erase(value)`, `find(value)` (returns an iterator to the element, or `end()` if absent), and `count(value)` (0 or 1 for a set; can exceed 1 for a multiset). The Topic 5.5 program inserts several integers (including a duplicate) into a set, shows the duplicate is silently ignored when the sorted, unique contents are printed, uses `find()` to confirm a value's presence, then repeats a similar sequence with a multiset to show `count()` correctly reporting more than one occurrence of a repeated value.

Q5. [5 marks] Explain the map class as a key-value container, with its syntax, common operations, and a complete program.

`map` (from `<map>`) stores key-value pairs with unique keys, automatically kept sorted by key (typically via a balanced binary search tree), giving $O(\log n)$ insertion, lookup and deletion by key — conceptually an associative array or dictionary. `multimap` is the equivalent that allows multiple values to share the same key. Elements can be inserted or updated using the `[]` operator (`m[key] = value;`) or explicitly with `insert({key,value});`, looked up with `find(key)` (returning an iterator, or `end()` if absent) or `count(key)`, and the whole map traversed in sorted-key order with a range-based for loop over pairs (`auto &p; : m`), where `p.first` is the key and `p.second` is the value. The Topic 5.6 program builds a map from roll numbers to student names, inserted out of numeric order, and shows the traversal automatically prints them sorted by roll number, followed by a `find()` lookup and an `erase()` by key.

Q6. [5 marks] Explain iterators in STL: categories, syntax, and the use of auto and reverse iterators, with a program.

An iterator is an object that behaves like a pointer, used to traverse a container's elements via `begin()` (first element) and `end()` (one past the last) without exposing the container's internal structure, so the same `*it / ++it` syntax works across very different container implementations. STL defines several iterator categories with increasing capability — input, output, forward, bidirectional, random-access — and a container only supports the operations its category allows; for example, `vector` supports full random-access iterator arithmetic (`it + 5`), while `list` supports only forward/backward stepping, not arbitrary jumps. The keyword `auto` lets the compiler deduce the (often long) iterator type automatically, and `rbegin()/rend()` provide reverse iterators that traverse a container back to front; the Topic 5.7 program demonstrates both a forward loop using `vector<int>::iterator` explicitly and a reverse loop using `auto` with `rbegin()/rend()`.

C. Ten-Mark / Long-Answer Questions

Q1. [10 marks] Explain the Standard Template Library in detail: its motivation, four components, and the generic programming philosophy behind it, with example programs for at least two containers.

Before STL, programmers routinely hand-wrote linked lists, dynamic arrays, trees and sorting routines for every project, duplicating effort and introducing bugs each time; STL (Topic 5.1) solves this by providing generic, template-based, thoroughly tested containers and algorithms that work with any data type satisfying the required operations (e.g. comparability for `sort()`). Explain all four components — containers (data storage, e.g. `vector`, `set`, `map`), algorithms (generic operations such as `sort()`, `find()`, `accumulate()` that work through iterators, not tied to a specific container), iterators (the uniform, pointer-like interface that lets any algorithm traverse any compatible container), and functors (objects overloading `operator()` used to customise algorithm behaviour) — and how they interoperate: an algorithm is written once against the iterator interface and automatically works with every container that exposes compatible iterators. Illustrate with the `vector sort/erase` program (Topic 5.3) and the `map student-record` program (Topic 5.6), explicitly pointing out where each of the four components appears in the code.

Q2. [10 marks] Compare and contrast all major STL containers covered in this chapter — `vector`, `list`, `set`, `multiset`, `map`, `multimap` — across ordering, duplicate handling, typical access pattern and underlying data structure, with a program for each category (sequence vs associative).

Present the full container comparison table from Topic 5.9/5.13: `vector` and `list` are sequence containers (insertion-order, user-controlled), while `set`/`multiset`/`map`/`multimap` are associative containers (automatically sorted by key, usually via a balanced BST, giving $O(\log n)$ operations). `vector` gives $O(1)$ random access but $O(n)$ middle-insertion, backed by a dynamic array; `list` gives $O(1)$ insertion/removal anywhere (given a position) but no indexing, backed by a doubly linked list — the `vector-vs-list` table in Topic 5.11 should be reproduced and explained with a concrete scenario for each (e.g. `vector` for a frequently-indexed list of scores, `list` for a playlist with frequent reordering). `set` enforces uniqueness and automatic sorting, useful for membership testing and removing duplicates; `map` pairs this with key-value storage, useful as a sorted dictionary; `multiset`/`multimap` relax the uniqueness constraint. Provide one complete worked program demonstrating a sequence container (`vector`, Topic 5.3) and one demonstrating an associative container (`map`, Topic 5.6), each with full traced output.

Q3. [10 marks] Discuss iterators and algorithms in STL comprehensively: iterator categories, the relationship between algorithms and iterators, and at least four different algorithms from and , each demonstrated with a program.

Explain that an iterator generalises the idea of a pointer, letting algorithms traverse any container uniformly via `begin()/end()`, with five increasingly capable categories (input, output, forward, bidirectional, random-access) determining which operations a given container's iterators support — `vector/array` support the most capable (random-access), `list` only bidirectional, and some containers only forward iteration. Because algorithms operate purely through iterators rather than any container-specific API, the same `sort()`, `find()`, `count()`, `max_element()`, `min_element()` and `accumulate()` work unchanged across `vector`, `deque`, `array`, and (where their iterator category permits) other containers too. Present the Topic 5.8 program in full, explaining `sort(v.begin(),v.end())` for ordering, `max_element()/min_element()` (dereferenced with `*` since they return iterators, not values) for extremes, `accumulate(v.begin(),v.end(),0)` from `<numeric>` for summation, and `find(v.begin(),v.end(),value)` for searching, including how `(it - v.begin())` converts an iterator back into a numeric position for a random-access container.

Q4. [10 marks] Explain the `set`, `multiset`, `map` and `multimap` classes in depth, including their internal ordering guarantee, typical time complexity, and when each is the most appropriate choice, with complete programs for `set` and `map`.

All four are associative containers, conventionally implemented as self-balancing binary search trees (e.g. red-black trees), guaranteeing elements are always visited in sorted order when traversed, and giving $O(\log n)$ time for insertion, lookup (`find()/count()`) and deletion — a significant advantage over a sequence container's $O(n)$ linear search when frequent lookups by value/key are needed. `set` stores unique elements directly (no associated value), ideal for membership testing or de-duplicating a collection; `multiset` relaxes uniqueness, useful when counting occurrences matters (demonstrated by `ms.count(2)` in Topic 5.5). `map` associates a unique key with a value, ideal for dictionary/lookup-table use cases such as mapping roll numbers to names (Topic 5.6); `multimap` relaxes the key-uniqueness constraint, useful when one key can legitimately have several associated values (e.g. a subject mapped to multiple student names). Provide the complete `set` program from Topic 5.5 and the complete `map` program from Topic 5.6, explaining in each case why the chosen container is the natural fit for its scenario.

Q5. [10 marks] Write and explain a complete C++ program that uses at least three different STL containers together (e.g. `vector`, `map`, `set`) to solve a realistic small problem (such as processing student records), including sorting, searching and aggregate computation using STL algorithms.

Design a program that reads a vector of Student-like records (or parallel `vector/struct` data), inserts each student's roll number and name into a `map` for quick name lookup by roll number, and separately inserts all distinct grades achieved into a `set` to report the unique set of grades present. Use `sort()` (from `<algorithm>`) on the vector of marks to order them, `accumulate()` (from `<numeric>`) to compute the total and hence average marks, and `max_element()/min_element()` to report the topper's and the lowest mark, printing all results via range-based for loops over each container. The explanation should justify each container choice (`vector` for the ordered list of marks since positional processing/sorting is needed, `map` for $O(\log n)$ name lookup by roll number, `set` for automatically de-duplicated, sorted grade reporting), tying together Topics 5.3, 5.5, 5.6 and 5.8 into one cohesive, examinable program.

D. Important Programming Questions

PRACTICE: WRITE THE FULL PROGRAM

- Write a C++ program to remove duplicate elements from a vector of integers using a set.
- Write a C++ program to count the frequency of each word in a sentence using a map.
- Write a C++ program that reads n integers into a vector, sorts them in descending order (using `sort()` with a custom comparator or `greater()`), and prints the result.
- Write a C++ program using a multimap to store multiple phone numbers against the same contact name, and print all numbers for a given name.
- Write a C++ program that uses a list to implement a simple queue-like task scheduler, adding tasks at the back and removing from the front.

E. Output Prediction Questions

Q1. Predict the output:

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s = {5, 1, 5, 3, 1, 4};
    for (int x : s) cout << x << " ";
    return 0;
}
```

ANSWER

```
1 3 4 5
```

Explanation: set stores only UNIQUE elements and keeps them automatically sorted in ascending order, regardless of insertion order or how many times a duplicate was listed — so {5,1,5,3,1,4} becomes {1,3,4,5}.

Q2. Predict the output:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30};
    cout << v.size() << " " << v.at(1) << " ";
    v.pop_back();
    cout << v.size();
    return 0;
}
```

ANSWER

```
3 20 2
```

Explanation: `v.size()` is initially 3; `v.at(1)` returns the element at index 1, which is 20; `v.pop_back()` removes the last element (30), leaving size 2.

Q3. Predict the output:

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> m;
    m["b"] = 2;
    m["a"] = 1;
    m["c"] = 3;
    for (auto &p : m) cout << p.first << p.second << " ";
    return 0;
}
```

ANSWER

```
a1 b2 c3
```

Explanation: map automatically keeps its elements sorted by KEY (here, alphabetically by string), regardless of the order they were inserted in — "b" was inserted first but "a" is printed first.

F. Find-the-Error Questions

Q1. Identify and correct the error:

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> l = {1, 2, 3};
    cout << l[1];           // ERROR
    return 0;
}
```

EXPLANATION

list does NOT support the [] (subscript) operator, because it is a doubly linked list with no O(1) random access. Fix: use an iterator (advance(l.begin(),1)) or std::next(l.begin(),1) and dereference it, e.g. *next(l.begin(),1).

Q2. Identify and correct the error:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {3, 1, 2};
    sort(v);           // ERROR
    return 0;
}
```

EXPLANATION

sort() requires a RANGE given as two iterators, not the container itself. Fix: sort(v.begin(), v.end());

Q3. Identify and correct the error:

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s = {1, 2, 3};
    s.push_back(4);    // ERROR
    return 0;
}
```

EXPLANATION

set has no push_back() (that is a sequence-container operation, e.g. for vector/list). Associative containers like set use insert() instead. Fix: s.insert(4);

G. Multiple Choice Questions (with explanations)

1. Which of these is NOT one of the four main STL components?

(A) Containers (B) Algorithms (C) Compilers ✓ (D) Iterators

Answer: (C). The four STL components are containers, algorithms, iterators, and functors (function objects) — "compilers" is not part of STL.

2. Which container gives O(1) random access by index?

(A) list (B) set (C) vector ✓ (D) map

Answer: (C). vector uses contiguous memory (a dynamic array), giving O(1) access by index via the [] operator, unlike list, set, or map.

3. What does container.end() return?

(A) An iterator to the last element (B) An iterator one position PAST the last element ✓ (C) The number of elements (D) A null iterator

Answer: (B). end() returns an iterator just past the last valid element; it must never be dereferenced, and ranges are conventionally written as [begin, end).

4. Which container does NOT allow duplicate elements?

(A) multiset (B) vector (C) set ✓ (D) multimap

Answer: (C). set enforces unique elements; inserting a duplicate value has no effect. multiset and multimap explicitly allow duplicates.

5. What is the typical time complexity of find() on a set?

(A) O(1) (B) O(n) (C) O(log n) ✓ (D) O(n log n)

Answer: (C). set is typically implemented as a balanced binary search tree, giving O(log n) search, insertion and deletion.

6. Which header provides the accumulate() algorithm?

(A) <vector> (B) <algorithm> ✓ (C) <set> (D) <map>

Answer: (B). accumulate() is declared in <numeric>, separate from the general header.

7. What does auto do when used to declare an iterator?

(A) Converts the iterator to an int (B) Lets the compiler deduce the iterator's type automatically ✓ (C) Makes the iterator constant (D) Has no effect

Answer: (B). auto lets the compiler infer the (often verbose) iterator type from the initialiser, improving readability without changing behaviour.

8. Which operation correctly adds an element to a set named s?

(A) s.push_back(x); (B) s.append(x); (C) s.insert(x); ✓ (D) s.add(x);

Answer: (C). Associative containers like set use insert(), not push_back() (which is for sequence containers such as vector/list/deque).

9. In a map, what does p.first and p.second refer to when iterating with auto &p; : m?

(A) p.first is the value, p.second is the key (B) p.first is the key, p.second is the value ✓ (C) Both refer to the same thing (D) They are undefined

Answer: (B). Each element of a map is a pair; first holds the key and second holds the associated value.

10. Which container is best suited for frequent insertion/removal in the MIDDLE of a sequence?

(A) vector (B) list ✓ (C) array (D) set

Answer: (B). list (a doubly linked list) supports $O(1)$ insertion/removal anywhere given an iterator to the position, unlike vector, which needs $O(n)$ shifting.

H. Viva-Voce Questions

BE READY TO ANSWER VERBALLY, IN YOUR OWN WORDS

- Why is STL described as "generic programming"?
- What is the practical difference between a sequence and an associative container?
- Why does vector give faster random access than list?
- Why does list not support the `[]` operator?
- What happens if you insert a duplicate value into a set?
- How does a multiset differ from a set, with an example?
- How is a map different from an array in terms of indexing?
- Why are algorithms like `sort()` written to take iterators rather than containers directly?
- What is the risk of erasing an element from a container while iterating over it carelessly?
- What does `begin()` and `end()` define, and why is `end()` "one past" the last element rather than AT it?
- What is a functor, and why would you use one with an algorithm like `sort()`?
- What is the time complexity difference between search in a vector versus a set?
- Why would you choose a map over a vector of pairs for lookup-heavy code?
- What iterator category does list support, and what does that rule out?
- Give a real scenario where you would prefer `multimap` over `map`.

I. Chapter 5 — Revision Notes (Must-Remember Points)

TOP 10 EXAM-CRITICAL POINTS

- STL = Containers + Algorithms + Iterators + Functors, all template-based and generic.
- vector: dynamic array, $O(1)$ indexed access, $O(n)$ middle insert/erase, best cache performance.
- list: doubly linked list, $O(1)$ insert/erase anywhere (given a position), NO `[]` indexing.
- set/multiset: unique/duplicate-allowing elements, automatically sorted, $O(\log n)$ operations (balanced BST).
- map/multimap: unique/duplicate-key-allowing key-value pairs, automatically sorted by key, $O(\log n)$ operations.
- `begin()` points to the first element; `end()` points ONE PAST the last — never dereference `end()`.
- Algorithms (`sort`, `find`, `count`, `accumulate`, `max_element`, `min_element`) operate through iterators, not containers directly.
- `accumulate()` lives in `<numeric>`, not `<algorithm>`.
- Associative containers use `insert()`, NOT `push_back()`; sequence containers use `push_back()/push_front()`.
- `at(i)` bounds-checks and throws `out_of_range`; `[]` does NOT bounds-check (undefined behaviour on an invalid index).